

SecureLink: lessons from the 2024 Snowflake customer data theft campaign

A secure messaging, attack simulation, and monitoring platform that applies course topics to a real cloud credential-abuse incident.

Name	Mitra Boga
Location	Hyderabad, India
edX	_mitraboga
GitHub	mitraboga
Date	May 5th, 2026
Incident	Snowflake customer data theft campaign / UNC5537
Incident date	June 2024
CVE	Not applicable / N/A





The Snowflake campaign was not mainly a cryptography failure. It was a layered security failure.



Identity

Attackers used valid customer credentials, many taken from infostealer malware logs.

Access control

Impacted accounts lacked MFA, credential rotation, or trusted-location restrictions.

Detection

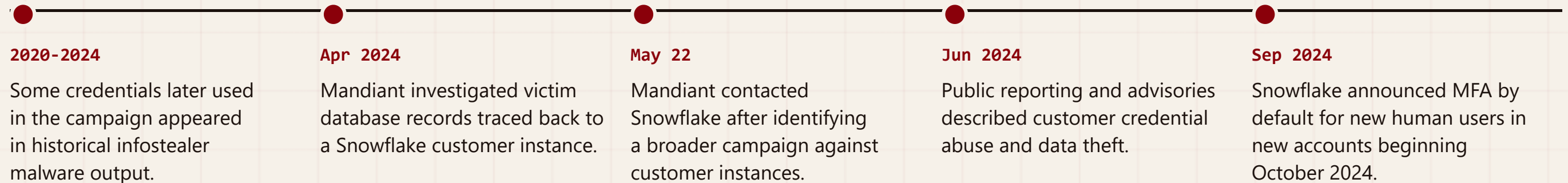
The campaign shows why abnormal access, large exports, and suspicious tooling must be visible quickly.

SecureLink uses this incident as a design pressure test: protect messages, verify integrity, reject replay and forgery, and make attacks observable.



How the campaign unfolded

Mandiant attributed the activity to UNC5537, a financially motivated threat cluster focused on theft, sale, and extortion of data from Snowflake customer environments.





Four missed controls turned stolen passwords into data theft.

01 Single-factor authentication

Valid username and password were enough for affected accounts that had not enforced MFA.

02 Stale credentials

Credentials stolen by infostealers remained usable, in some cases long after the original compromise.

03 No network allow lists

Customer instances were not limited to trusted source locations, increasing the value of stolen credentials.

04 Weak anomaly detection

Large exports, unusual client applications, VPN infrastructure, and unfamiliar access patterns needed faster alerting.



UNC5537 followed a simple but effective path: valid login, query, stage, export, extort.

Credential source

Infostealer logs and previously exposed credentials supplied Snowflake account access.

>

Access

Accounts without MFA allowed password-only authentication from attacker-controlled infrastructure.

>

Recon

Commands such as SHOW TABLES identified databases and tables of interest.

>

Stage

COPY INTO placed selected data into temporary stages, often compressed.

>

Exfiltrate

GET downloaded staged data, followed by attempted sale or extortion.

Course connection: this maps directly to securing accounts, access controls, auditing, incident response, and privacy preservation.



CS50 topics used in this project

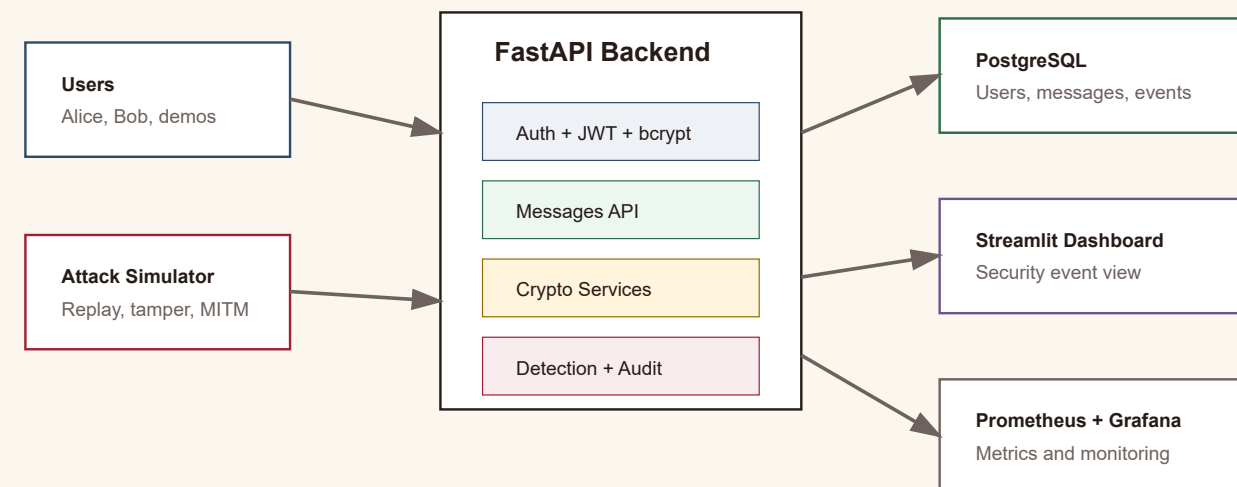
CS50 THEME	INCIDENT LESSON	SECURELINK IMPLEMENTATION
Securing accounts	Stolen credentials and missing MFA enabled access.	bcrypt password hashing, JWT auth, protected routes, failed-login telemetry.
Securing data	Cloud-hosted customer data became the target.	AES-256-GCM encryption, HMAC-SHA256 integrity checks, RSA-PSS signatures.
Securing systems	Trusted-location controls and telemetry mattered.	Caddy TLS proxy, Redis rate limiting, readiness checks, Prometheus and Grafana.
Securing software	Abuse must be testable, detectable, and logged.	Replay, tamper, invalid-signature, MITM, and brute-force simulations.
Preserving privacy	Compromise of a data platform can expose many downstream users.	Encrypted storage, authenticated access, audit events, and limited plaintext exposure.



SecureLink is a working security system, not only a slideshow example.

SecureLink Architecture

Encrypted messaging, attack detection, audit logging, and monitoring



Backend

FastAPI services handle auth, keys, messages, validation, detection, and event logging.

Data layer

SQLAlchemy with PostgreSQL or SQLite stores users, encrypted messages, and security events.

Operations

Docker Compose, Caddy, Redis, Streamlit, Prometheus, and Grafana support a production-style demo.



SecureLink protects each message before it is stored or read.

Send path

POST /messages/send

Authenticate the sender, encrypt the plaintext with AES-256-GCM, compute HMAC-SHA256, sign metadata with RSA-PSS, then store ciphertext and audit data.

Read path

GET /messages/inbox

Check JWT, reject replayed IDs or nonces, verify HMAC and signature, then decrypt only after the message passes validation.

AES-GCM

Confidentiality and authenticated encryption for message content.

HMAC-SHA256

Integrity signal that detects tampering before trusting stored data.

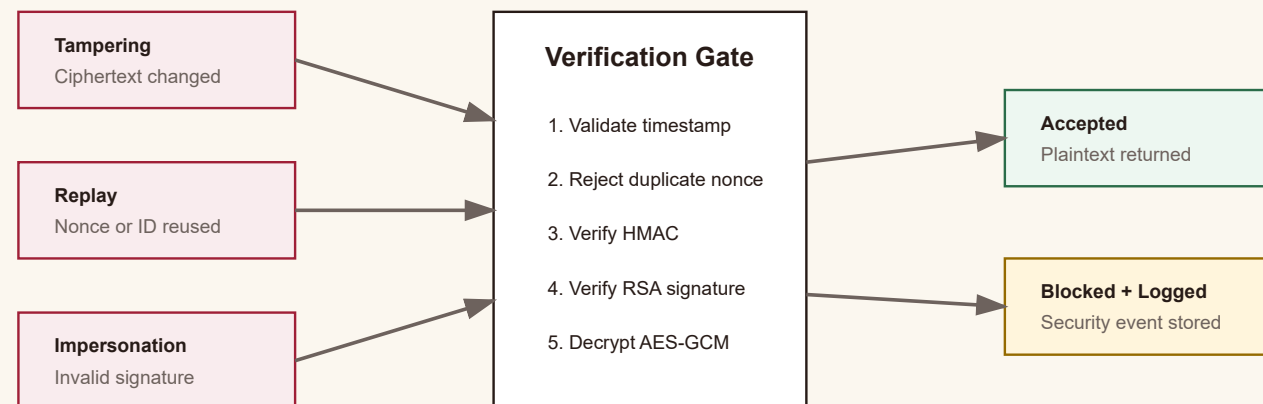
RSA-PSS + DH

Digital signatures and key exchange demonstrations connect to public-key cryptography.



The project includes attacks so defenses can be demonstrated, not only described.

Attack Flow and Defenses



Replay

Duplicate message IDs, nonces, or stale timestamps are rejected and logged.

Tamper

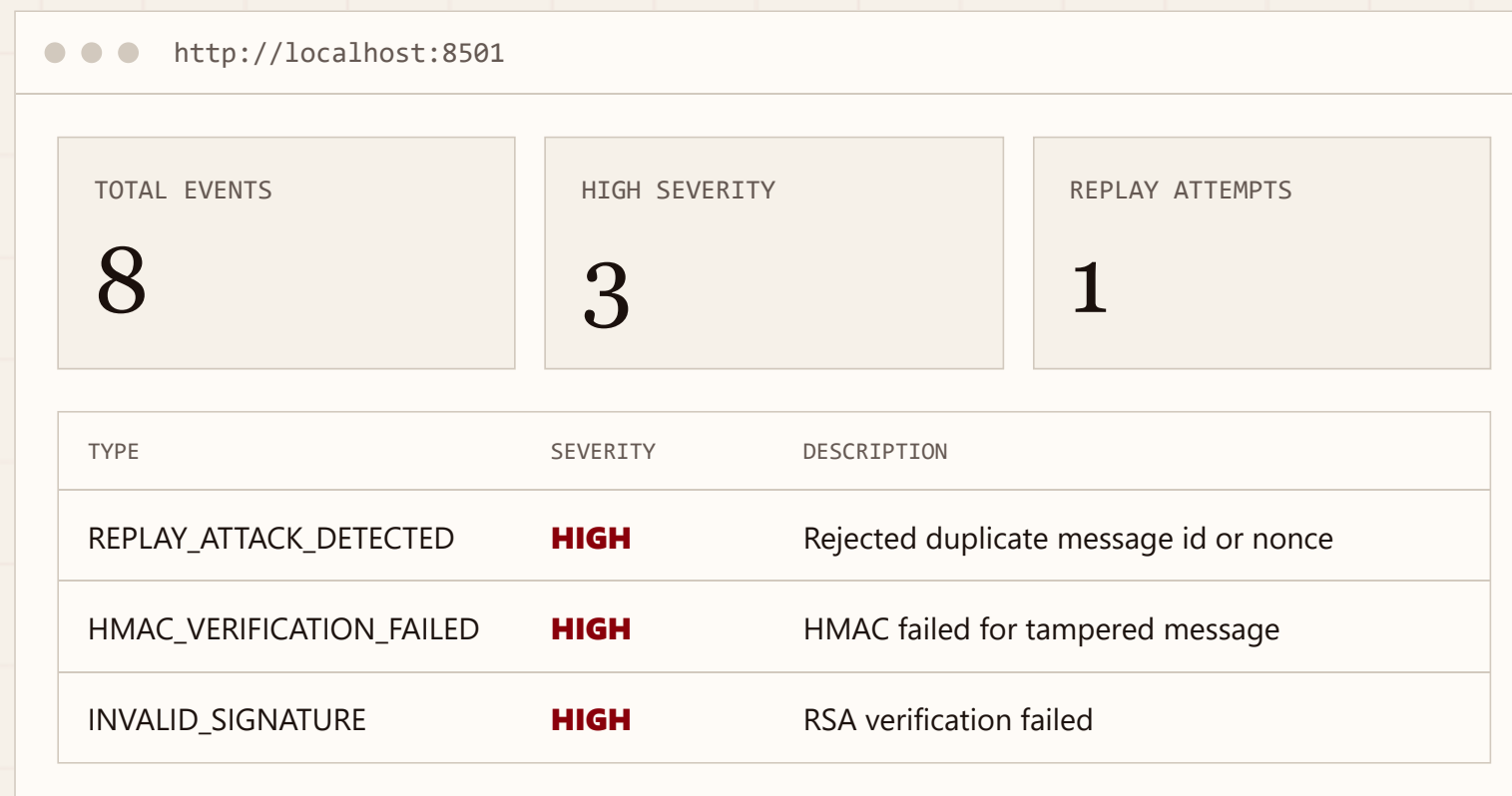
Modified ciphertext fails HMAC or AES-GCM validation before plaintext is returned.

Invalid signature

Forged metadata fails RSA-PSS verification and becomes a security event.



Monitoring is the bridge between prevention and incident response.



Streamlit

Shows security event counts, severity breakdowns, and recent events.

Prometheus

Scrapes API metrics from `/metrics` for operational visibility.

Grafana

Presents production-style monitoring so unusual behavior is visible during demos.



How others can avoid the same outcome

01 Enforce MFA or passkeys

Make single-factor sign-ins unavailable for human users, especially administrators and data access roles.

03 Restrict trusted locations

Use network policies, device posture, and least privilege for high-value data platforms.

05 Protect data in layers

Use encryption, integrity checks, signatures, replay controls, and strong secrets management.

02 Rotate exposed credentials

Monitor infostealer dumps and treat old exposed passwords as active risk until rotated.

04 Alert on abnormal access

Detect unusual clients, VPN/VPS infrastructure, new locations, large exports, and staging commands.

06 Practice incident response

Run attack simulations and verify that events reach dashboards, logs, and responsible humans.



SecureLink turns the Snowflake lesson into a concrete engineering pattern.

What the incident taught

Cloud data security depends on account security, access policy, monitoring, and response. Strong encryption alone cannot save an environment where valid credentials are abused invisibly.

What the project demonstrates

SecureLink applies CS50 cybersecurity topics in a working system: authentication, cryptography, replay protection, attack simulation, audit logging, TLS, rate limiting, and observability.

Suggested recording flow: slides 1-6 explain the incident and course mapping; slides 7-10 show SecureLink architecture, crypto, attacks, and dashboard; slides 11-12 close with recommendations and limitations.